

RENSSELAER MECHATRONICS

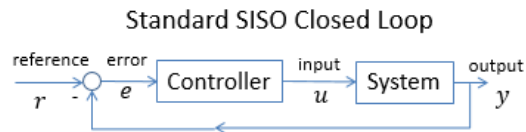
DC Motor Control with SISOtool: PD Compensator

Objectives:

- Design a SISO compensator to meet transient specifications
- Be aware a step response input is challenging to avoid saturation

Background Information:

The standard SISO controller uses the output compared to a reference



SISO Controller Design:

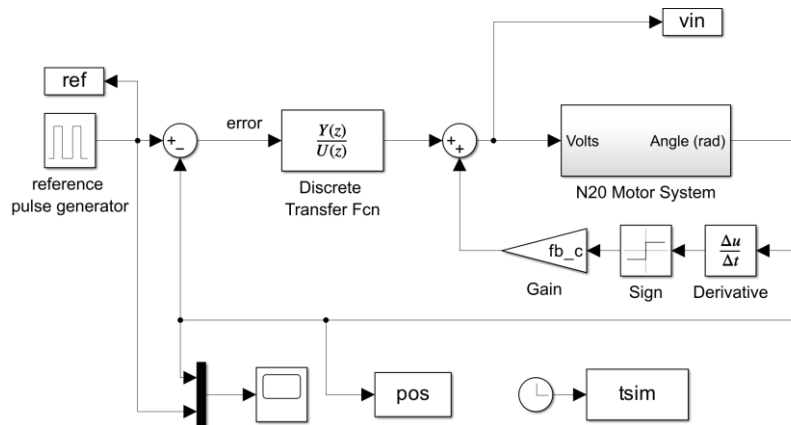
Design a standard SISO controller (using SISOTOOL in MATLAB) for the position control of a DC motor such that the closed loop poles satisfy a settling time of 0.25 seconds and 5% overshoot for a unit step response. Use a DC motor model with $K = 5.5$, $\tau = 0.045$. Ignore potential saturation (assume infinite control effort available) and other hardware limitations (use a PD compensator – no extra pole)

Questions:

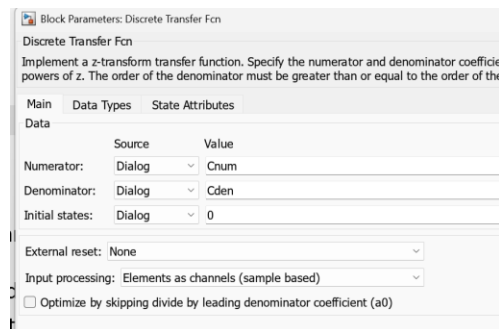
- What are the zeros, poles and gains of your compensator (in SISOTOOL choose 'Edit', 'SISOTOOL Preferences', 'Options', 'Zero/Pole/Gain')?
- Where are the closed loop poles and zeros located?
- What is the actual percent overshoot and settling time? If it is different than the designed overshoot, explain why.
- Write out the control law of your resulting compensator (multiply everything out):
 - $u =$
- What can you say about the magnitude of the control effort initially (based off the above control law)?

Implementation on Hardware: Discretizing the Compensator

Currently the compensator designed will not be able to be implemented as a transfer because it is not proper. It can be implemented if the transfer function is approximated as a discrete transfer function. Modify the Simulink diagram from the previous lab:



Use 'Cnum' and 'Cden' in the transfer function block so the controller can be specified from the script file.



Keep the previous solver settings as fixed-step discrete at 0.005. Also modify the previous script file that it will discretize the compensator from SISOtool (you will need to specify your gain k and zero z_c from SISOtool)

```

%% PD
rdes=2.2; % desired reference value
s=tf('s');
Cs=k*(s+zc); % compensator from SISOtool
fb_c=0.4; % coulomb friction voltage estimate

% Discretize controller
TS=.005;
Cd=c2d(Cs, TS, 'tustin');
[num den]=tfdata(Cd);
Cnum=num{:}, Cden=den{:}

%% Open the simulink diagram and run experiment

close all
plot(tsim, pos), hold on
plot(tsim, ref, 'r-')

% Use logged simulation time and reference
% to calculate predicted response based on model:
% plant transfer function G needs to be defined
[y t]=lsim(feedback(Cs*G,1),ref,tsim);
plot(t,y, 'k--')
legend('experiment', 'reference', 'theory')

```

Run the first section to discretize your compensator. Run the experiment with the Simulink diagram. Then run the last part of the code to plot the results.

You may notice the system does not function at all trying to implement the pure PD compensator. We know that is requesting infinite control effort. Plot the control effort to get some insight to what is happening.

To help mitigate this a compensator pole can be used to make an approximate PD compensator (lead compensator) and we can reduce the desired reference.

Questions:

For each of the following test conditions provide a screenshot of the whole response, and a single step response

- Reference $r_{des}=2.2$ and PD compensator: $k(s + z_c)$
 - What does the control effort look like?
- Reference $r_{des}=2.2$ and PD compensator with a pole at -1000: $k(s + z_c)/(0.001s + 1)$
- Reference $r_{des}=2.2$ and PD compensator with a pole at -100: $k(s + z_c)/(0.01s + 1)$
- Reference $r_{des}=0.7$ and PD compensator with a pole at -100: $k(s + z_c)/(0.01s + 1)$

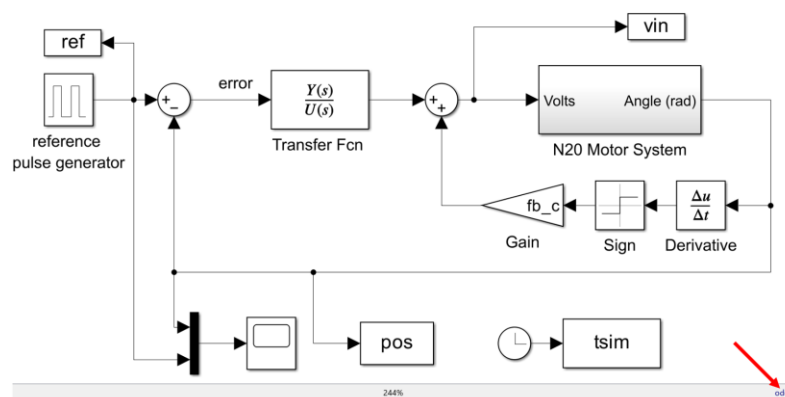
In the above experiments you can use the same k and z_c for all the experiments - just modify the compensator as shown above. If you examine these in SISOtool you may notice that closed loops poles will move a little of the desired location, but that is okay for this exercise. It is easier than recomputing the gain and adjusting the zero location to match exactly.

You should notice that as the compensator and reference change to cause less saturation the results will match closer to the ideal transfer function-based model. Of course, we could always simulate the effect of saturation and coulomb friction in Simulation, but this exercise it trying to get conditions closer to the ideal model.

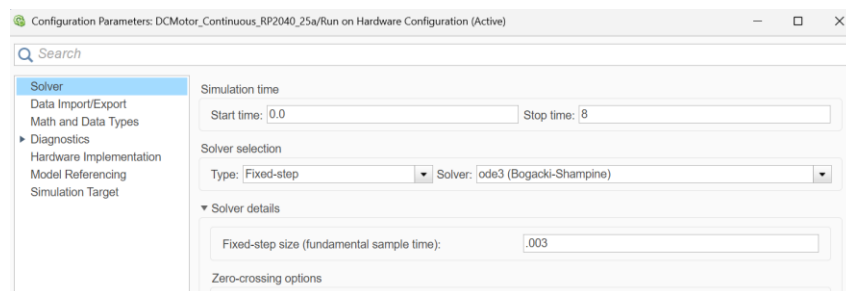
Implementation on Hardware: Use a Fixed Step Solver and a continuous time transfer function

In the previous section the compensator was discretized so it could be implemented with a fixed-step discrete solver. In this section a continuous time transfer function can be implemented on hardware by selecting a fixed step ode solver to be used on the hardware.

Modify your previous simulation diagram as follows:



Replace the discrete transfer function block with a normal transfer function block. In the solver settings select 'fixed-step' and 'ode3'.



Selecting a fixed step solver allows any transfer function blocks to be approximated by the solver (as opposed to directly approximated as in the previous section). This can be easier, but you might not know exactly how it is being approximated and must rely on how the solver method is doing this. In the previous section the discretization is exactly specified/known.

With this method you cannot implement a pure PD compensator, so skip that test and run the PD compensator with a pole at -1000 first.

```

%% PD
rdes=2.2; % desired reference value
s=tf('s');
Cs=k*(s+zc)/(.001*s+1) % compensator from SISOtool
fb_c=0.4; % coulomb friction voltage estimate

% extract num, den from the TF
[num den]=tfdata(Cs);
Cnum=num{:}, Cden=den{:}

%% Open the simulink diagram and run experiment:

close all
plot(tsim, pos), hold on
plot(tsim, ref, 'r-')

% Use logged simulation time and reference
% to calculate predicted response based on model:
% plant transfer function G needs to be defined
[y t]=lsim(feedback(Cs*G,1),ref,tsim);
plot(t,y, 'k--')
legend('experiment','reference','theory')

```

Questions:

For each of the following test conditions provide a screenshot of the whole response, and a single step response zoomed in around the first step that occurs at time 0.

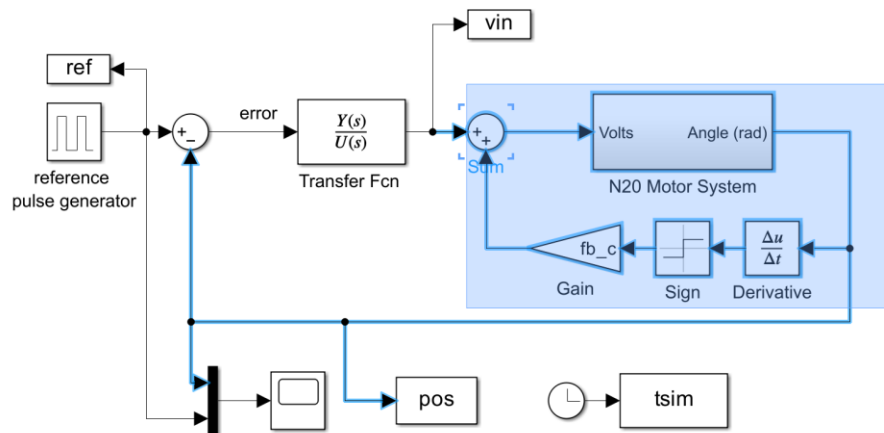
- Reference $r_{des}=2.2$ and PD compensator: $k(s + z_c)$ (won't run – no graph needed)
- Reference $r_{des}=2.2$ and PD compensator with a pole at -1000: $k(s + z_c)/(0.001s + 1)$ (will not run well)
- Reference $r_{des}=2.2$ and PD compensator with a pole at -100: $k(s + z_c)/(0.01s + 1)$
- Reference $r_{des}=0.7$ and PD compensator with a pole at -100: $k(s + z_c)/(0.01s + 1)$

The last two results should be very similar to the experiments in the previous section.

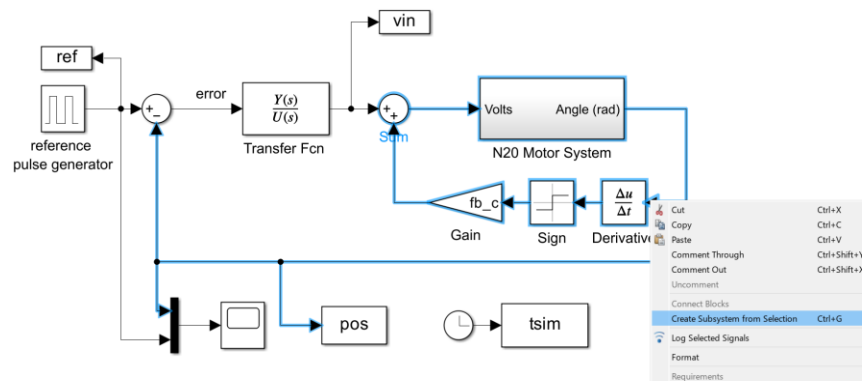
Notice that approximating use ODE3 was not successful for a compensator pole at -1000. It is important to know under what circumstances you will be able to successfully resolve poles in your compensator. This topic will be discussed in future labs.

Simplifying the hardware motor subsystem:

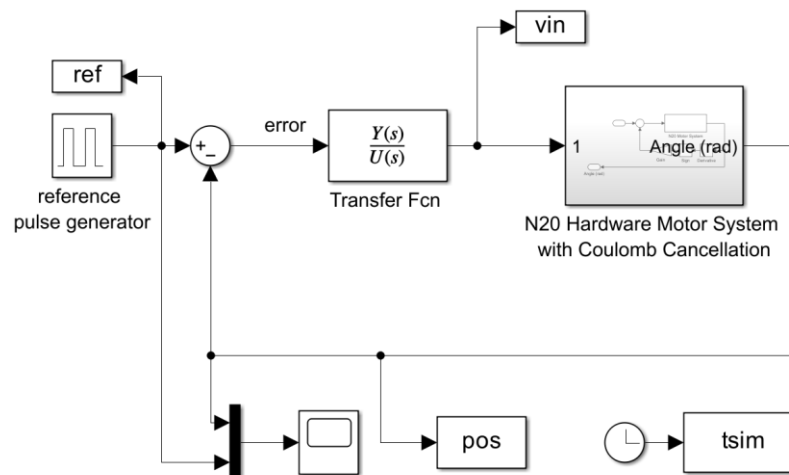
To make the hardware system easier to use in future labs combine the N20 motor subsystem with the feedback linearization. First move the logging of 'vin' to directly after the transfer function, then select the motor subsystem, the summing junction, and the coulomb feedback term:



Right-click and create subsystem:



Rename this subsystem so you know it has the coulomb friction cancellation inside:



Run your final compensator code from to verify this code still runs as expected. This block will be used in the next lab.